

Products – Docs

The **products** endpoint provides a comprehensive dataset of sample product information, including details like names, prices, descriptions, images, and categories, ideal for testing and prototyping e-commerce applications.

Get all products

By default you will get 30 items, use [Limit and skip](#) to paginate through all items.

```
fetch('https://dummyjson.com/products')  
.then(res => res.json())  
.then(console.log);
```

Show Output

DummyJSON



Get a single product

```
fetch('https://dummyjson.com/products/1')  
.then(res => res.json())  
.then(console.log);
```

Show Output

Search products

```
fetch('https://dummyjson.com/products/search?q=phone')  
.then(res => res.json())  
.then(console.log);
```

Limit and skip products

You can pass `limit` and `skip` params to limit and skip the results for pagination, and use `limit=0` to get all items.

You can pass `select` as query params with comma-separated values to select specific data

```
fetch('https://dummyjson.com/products?limit=10&skip=10&select=title,
.then(res => res.json())
.then(console.log);
```

Show Output

DummyJSON



Sort products

You can pass `sortBy` and `order` params to sort the results, `sortBy` should be field name and `order` should be "asc" or "desc"

```
fetch('https://dummyjson.com/products?sortBy=title&order=asc')
  .then(res => res.json())
  .then(console.log);
```

Hide Output

```
{
  "products": [
    {
      "id": 167,
      "title": "300 Touring", // sorted by title in ascending order
      "price": 28999.99
      /* rest product data */
    },
    {
      "id": 99,
      "title": "Amazon Echo Plus", // sorted by title in ascending order
      "price": 99.99
      /* rest product data */
    },
    {...}
  ],
  // 30 items
  "total": 194,
  "skip": 0,
  "limit": 30
}
```

Get all products categories

```
fetch('https://dummyjson.com/products/categories')
  .then(res => res.json())
```

DummyJSON



Get products category list

```
fetch('https://dummyjson.com/products/category-list')  
  .then(res => res.json())  
  .then(console.log);
```

Show Output

DummyJSON



Get products by a category

```
fetch('https://dummyjson.com/products/category/smartphones')
  .then(res => res.json())
  .then(console.log);
```

Show Output

Add a new product

Adding a new product will not add it into the server.
It will simulate a POST request and will return the new created product with a new id

```
fetch('https://dummyjson.com/products/add', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
```

DummyJSON



```
//  
.then(res => res.json())  
.then(console.log);
```

Show Output

Update a product

Updating a product will not update it into the server.

It will simulate a PUT/PATCH request and will return updated product with modified data

```
/* updating title of product with id 1 */  
fetch('https://dummyjson.com/products/1', {  
  method: 'PUT', /* or PATCH */  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({  
    title: 'iPhone Galaxy +1'  
  })  
})  
.then(res => res.json())  
.then(console.log);
```

Show Output

DummyJSON



Deleting a product will not delete it into the server.

It will simulate a DELETE request and will return deleted product with `isDeleted` & `deletedOn` keys

```
fetch('https://dummyjson.com/products/1', {  
  method: 'DELETE',  
})  
.then(res => res.json())  
.then(console.log);
```

Show Output

Buy me a coffee



in